

L LINKORA

Professional Networking & Career Platform

"Connect. Grow. Succeed."

React Native (Expo) + Firebase

Contents

1. [Executive summary](#)
2. [Problem statement](#)
3. [Objectives](#)
4. [Proposed solution](#)
5. [Features](#)
6. [Technology stack](#)
7. [System architecture](#)
8. [Application workflow](#)
9. [Database design](#)
10. [Authentication design](#)
11. [Security](#)
12. [UI / UX — screens](#)
13. [Firebase services used](#)
14. [Data flow](#)
15. [Testing](#)
16. [Error handling](#)

17. [Performance](#)
18. [Infrastructure independence](#)
19. [Installation guide](#)
20. [Running the application](#)
21. [Deployment](#)
22. [Future enhancements](#)
23. [Conclusion](#)
24. [Viva / presentation questions](#)

1.Executive summary

Linkora is a cross-platform mobile application for professional networking. A user creates an account, maintains a professional profile, publishes updates to a shared feed, engages with other members' content through likes and comments, and builds a network through connection requests.

The application is built with **React Native** using the **Expo** toolchain, and uses **Firebase** as a complete serverless backend. There is no custom API server: the mobile client communicates directly with Firebase Authentication, Cloud Firestore and Firebase Storage over HTTPS, with all authorisation enforced server-side by Firebase Security Rules.

The current build implements authentication, profiles, the feed, posts with images, likes, comments, connections, in-app notifications and settings. Jobs, job applications, direct messaging and push notifications are designed but **not implemented**; they are documented as such throughout rather than presented as working features.

A defining constraint of this project is **infrastructure independence**: Linkora is a personal application, architected so that it cannot depend on any employer or organisational infrastructure. This is enforced in code, not merely stated as policy — see §18.

2.Problem statement

Professional networking platforms are centralised. A user's profile, posts, connections and message history are held by a third-party company under terms the user does not control.

1. **No data ownership.** The user cannot export, inspect or delete their own data on their own terms.
2. **Opaque ranking.** Feed content is decided by an algorithm optimised for the platform's engagement metrics.
3. **Advertising and profiling.** Professional data is monetised for targeted advertising.
4. **Platform dependency.** If the account is suspended, the professional network is lost.

A secondary problem motivated the specific architecture. When a personal application is developed on a workstation that also has access to organisational systems, there is a real risk of **accidental infrastructure coupling** — a hardcoded internal URL, a shared connection string, a corporate proxy setting, or a service account committed to the repository. Such coupling creates both a security exposure for the organisation and a functional dependency that breaks the personal application the moment it leaves the corporate network.

3.Objectives

PRIMARY

1. Build a functional cross-platform networking application from a single codebase.
2. Implement secure authentication where the application never stores or handles passwords.
3. Design a Firestore model for profiles, posts, likes, comments, connections and notifications.
4. Enforce authorisation on the server through Security Rules, not in client code.
5. Provide realtime updates so content appears without manual refresh.
6. Support image upload with server-enforced type and size limits.

ARCHITECTURAL

7. Confine all network access to a single auditable module.
8. Guarantee the application connects to exactly one Firebase project, verifiably.
9. Ensure it runs on any personal device and ordinary internet connection, with no dependency on organisational network, VPN, servers, databases or credentials.

ENGINEERING

10. Use TypeScript so data shapes and navigation routes are checked at compile time.
11. Keep the toolchain minimal enough to run without Android Studio, Xcode or a JDK.

4. Proposed solution

Linkora addresses the problem through a **serverless, client-direct architecture** on infrastructure owned by the individual developer.

Problem	How Linkora addresses it
No data ownership	All data resides in a Firebase project owned by the developer's personal Google account.
Opaque ranking	The feed is strictly reverse-chronological (<code>orderBy('createdAt', 'desc')</code>). No ranking algorithm exists.
Advertising and profiling	No analytics SDK, crash reporter, advertising library or third-party tracker is present in the dependency tree.
Platform dependency	The owner controls the project; no external moderator can suspend the account.
Accidental infrastructure coupling	Enforced in code: a single network entry point, a hardcoded project allow-list that halts the app on mismatch, and a verifiable bundle.

WHY SERVERLESS

A conventional three-tier design exists largely to hold credentials and enforce authorisation away from the client. Firebase Security Rules perform authorisation on Google's servers on every request. A middle tier would therefore add hosting cost, deployment complexity and an additional attack surface without adding a guarantee the rules do not already provide.

5.Features

5.1 Implemented WORKING

Feature	Description	Implementation
Registration	Email + password sign-up; creates Auth record and Firestore profile	<code>auth.service.ts</code> → <code>register()</code>
Login	Email + password authentication	<code>auth.service.ts</code> → <code>login()</code>
Persistent session	User stays signed in after closing the app	<code>getReactNativePersistence(AsyncStorage)</code>
Password reset	Reset email sent by Firebase	<code>requestPasswordReset()</code>
Logout	With confirmation dialog	<code>SettingsScreen</code> → <code>logout()</code>
User profile	Name, headline, location, bio, photo	<code>users.service.ts</code> , Profile/EditProfile screens
Profile photo upload	Gallery image stored in Firebase Storage	<code>uploadAvatar()</code>
Home feed	Realtime reverse-chronological, 50 most recent	<code>subscribeToFeed()</code>
Create post	Text (≤3000 chars) with optional image	<code>createPost()</code>
Delete post	Author only; removes likes and comments first	<code>deletePostCompletely()</code>
Likes	One per user per post, atomic counter	<code>likePost()</code> / <code>unlikePost()</code>
Comments	Realtime thread per post	<code>comments.service.ts</code>
Delete comment	Comment author or post author	<code>deleteComment()</code>
Connections	Send / accept / remove requests	<code>connections.service.ts</code>
People directory	Up to 50 members, ordered by name	<code>listOtherUsers()</code>

In-app notifications	Like, comment, request, accepted	<code>notifications.service.ts</code>
Other user profiles	View any member and their posts	<code>UserProfileScreen</code>
Settings	Account details plus live backend identification	<code>SettingsScreen</code>

5.2 Not implemented

Feature	Status	Reason
Direct messaging	NOT IMPLEMENTED	No screen, service or route. <code>conversations/messages</code> denied in rules.
Jobs	NOT IMPLEMENTED	Collection designed and denied in rules; no code.
Companies	NOT IMPLEMENTED	As above.
Job applications	NOT IMPLEMENTED	As above.
Push notifications (FCM)	NOT IMPLEMENTED	Needs native modules unavailable in Expo Go, plus a server to send messages.
Search	NOT IMPLEMENTED	No search UI. <code>findUserByEmail()</code> exists but no screen calls it.
Post editing	NOT IMPLEMENTED	Permitted by rules; no UI.
Account deletion	NOT IMPLEMENTED	Permitted by rules; no UI.
Automated tests	NOT IMPLEMENTED	No test runner; <code>npm test</code> does not exist.

Unused code, stated honestly. Three functions are defined but called by no screen: `findUserByEmail()`, `deleteNotification()` and `currentUser()`. They compile and are typed, but they do not represent working features.

6. Technology stack

Layer	Technology	Version	Purpose
Language	TypeScript	6.0.3	Typed application code
UI framework	React	19.2.3	Component model
Mobile framework	React Native	0.86.3	Native rendering on Android and iOS
Toolchain	Expo SDK	57.0.20	Build, run and device delivery without native IDEs
Backend SDK	Firebase JS SDK	12.18.0	Auth, Firestore, Storage clients
Authentication	Firebase Authentication	—	Email/password identity
Database	Cloud Firestore	—	Document database with realtime listeners
File storage	Firebase Storage	—	Profile and post images
Navigation	React Navigation	7.x	Native stack + bottom tabs
Local storage	AsyncStorage	2.2.0	Session persistence
Media	expo-image-picker	57.0.16	Gallery access
Screen primitives	react-native-screens	4.26.0	Native screen containers
Safe areas	react-native-safe-area-context	5.7.0	Notch / home-indicator insets

Notably absent by design: no analytics SDK, no crash reporting, no advertising library, no HTTP client such as `axios`, no state-management library, no backend framework, and no SQL or NoSQL server other than Firestore.

JUSTIFICATION OF KEY CHOICES

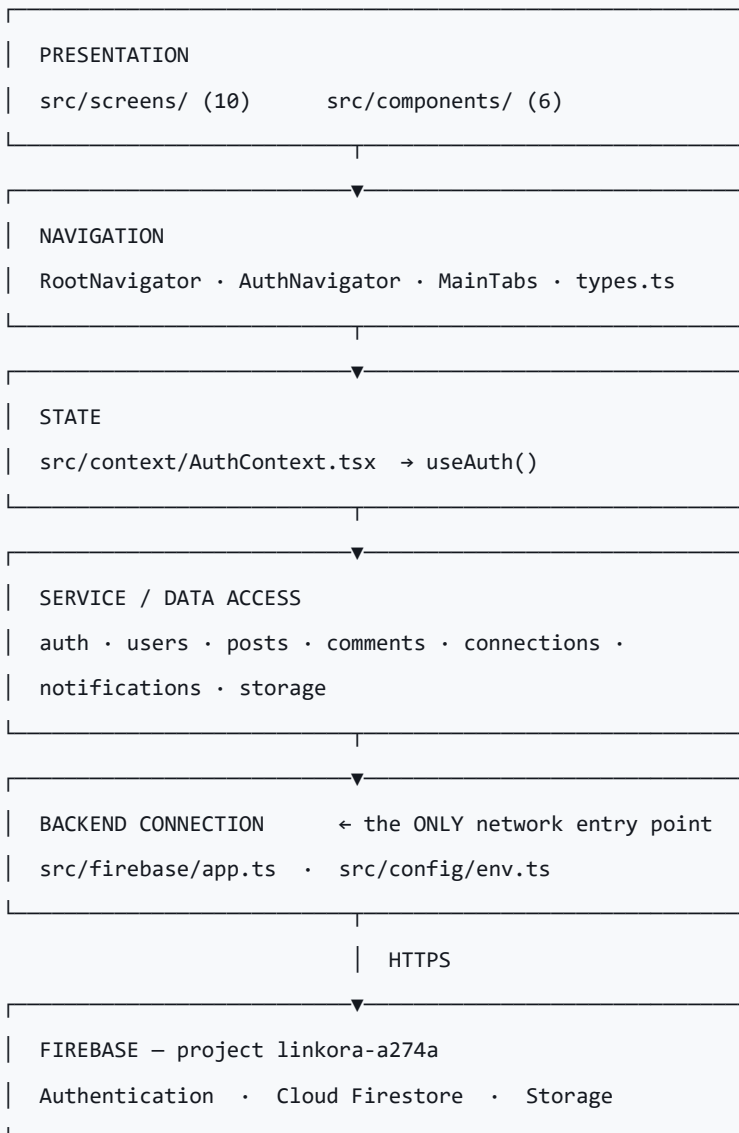
React Native + Expo. One codebase targets both platforms. Expo was chosen over bare React Native because it allows the application to run on a physical device through Expo Go without Android Studio, Xcode or a JDK — the development machine used for this project has none of these installed.

Firebase JS SDK over `@react-native-firebase`. The React Native Firebase library requires native modules and therefore a custom development build. The JS SDK runs inside Expo Go. The accepted

trade-off is the loss of Firebase Cloud Messaging, which is the direct reason push notifications are not implemented.

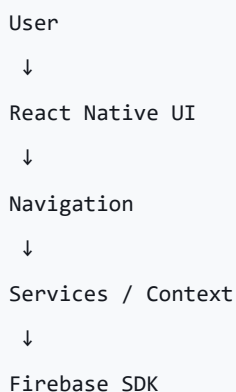
7. System architecture

7.1 Layered architecture



Dependencies point downward only. No service imports a screen; no screen imports the Firebase SDK.

7.2 Client–backend communication



↓

Firebase Authentication / Firestore / Storage

↓

Linkora Firebase project (linkora-a274a)

There is no REST API, no GraphQL endpoint and no custom server. Every operation is an SDK call that becomes an authenticated HTTPS request to a Google endpoint, carrying the user's Firebase ID token. That token is what populates `request.auth` in the security rules.

7.3 Project structure

Linkora/

```
├─ App.tsx · index.ts · app.json · package.json · tsconfig.json
├─ .env.example · .gitignore
├─ firebase.json · firestore.rules · firestore.indexes.json · storage.rules
├─ assets/ · docs/
├─ src/
│   ├─ components/  Avatar · Button · EmptyState · ErrorBanner · Input · PostCard
│   ├─ config/      env.ts
│   ├─ context/     AuthContext.tsx
│   ├─ firebase/    app.ts
│   ├─ navigation/  RootNavigator · AuthNavigator · MainTabs · types.ts
│   ├─ screens/     10 screens (2 under auth/)
│   ├─ services/    7 service modules
│   └─ theme/ · types/ · utils/
```

8.Application workflow

First launch

App.tsx

- SafeAreaProvider
- AuthProvider subscribes to onAuthStateChanged
- RootNavigator shows a spinner while initialising

```
|— no saved session → AuthNavigator → LoginScreen
```

```
└─ saved session    → MainTabs → HomeScreen
```

Registration

RegisterScreen

- ```
→ client validation (name ≥ 2, valid email, password ≥ 6, passwords match)
→ register()
 → createUserWithEmailAndPassword() Firebase Auth
 → updateProfile({ displayName }) Firebase Auth
 → createUserProfile() Firestore users/{uid}
→ onAuthStateChanged fires
→ AuthProvider sets user, opens profile listener
→ RootNavigator renders MainTabs
```

## Posting

HomeScreen "+ Post" → CreatePostScreen

- (optional) permission → launchImageLibraryAsync
- (optional) uploadPostImage() → Storage → download URL
- createPost() → addDoc(posts)
- feed listener pushes the new post to every signed-in device

## Engagement

```
Like → writeBatch{ set(likes/{uid}), update(likeCount +1) } → notification
```

```
Comment → addDoc(comments) → update(commentCount +1) → notification
```

```
Connect → setDoc(connections/{sortedPair}, 'pending') → notification
```

```
Accept → updateDoc(status 'accepted') → notification
```

## 9.Database design

```
users/{uid}
└─ notifications/{notificationId}

posts/{postId}
├─ likes/{likerUid}
└─ comments/{commentId}

connections/{connectionId} id = sorted(uidA, uidB).join('_')

— designed but NOT IMPLEMENTED —
jobs/{jobId} · companies/{companyId} · applications/{applicationId}
conversations/{conversationId}/messages/{messageId}
```

### 9.1 users/{uid}

| Field                 | Type      | Description                       |
|-----------------------|-----------|-----------------------------------|
| uid                   | string    | Equals the document ID            |
| email                 | string    | Sign-in address                   |
| displayName           | string    | Full name                         |
| headline              | string    | One-line professional description |
| bio                   | string    | Extended description              |
| location              | string    | City / country                    |
| photoURL              | string    | Storage download URL, or empty    |
| createdAt / updatedAt | timestamp | serverTimestamp()                 |

**Create:** owner, at registration. **Read:** any authenticated user. **Update:** owner only, restricted to presentation fields — `email` and `uid` are immutable from the client. **Delete:** owner (no UI).

### 9.2 posts/{postId}

| Field | Type | Description |
|-------|------|-------------|
|-------|------|-------------|

|                                              |           |                           |
|----------------------------------------------|-----------|---------------------------|
| authorId                                     | string    | Author UID                |
| authorName / authorHeadline / authorPhotoURL | string    | Denormalised author data  |
| text                                         | string    | Body, ≤ 3000 characters   |
| imageUr1                                     | string    | Storage URL, or empty     |
| likeCount / commentCount                     | number    | Maintained by increment() |
| createdAt                                    | timestamp | serverTimestamp()         |

#### DESIGN NOTE — DENORMALISATION

Author name, headline and photo are copied onto each post. Firestore has no join operation and bills per document read; storing only authorId would require 50 additional reads to render a 50-post feed. The cost is that a renamed user's historical posts retain the previous name.

### 9.3 posts/{postId}/likes/{likerUid}

Fields: uid (string), createdAt (timestamp).

#### DESIGN NOTE — IDENTITY AS KEY

The document ID is the liker's UID. This makes duplicate likes structurally impossible, removes the need for a pre-write existence query, and reduces the authorisation rule to a single identity comparison.

### 9.4 posts/{postId}/comments/{commentId}

Fields: authorId, authorName, authorPhotoURL, text (1–1000 characters), createdAt. Deletable by the comment author **or** the post author.

### 9.5 connections/{connectionId}

| Field       | Type            | Description            |
|-------------|-----------------|------------------------|
| members     | array[2] string | Both UIDs, sorted      |
| requesterId | string          | Initiator              |
| recipientId | string          | Recipient              |
| status      | string          | 'pending'   'accepted' |

|                                    |                        |                                |
|------------------------------------|------------------------|--------------------------------|
| <code>createdAt / updatedAt</code> | <code>timestamp</code> | <code>serverTimestamp()</code> |
|------------------------------------|------------------------|--------------------------------|

#### DESIGN NOTE — DETERMINISTIC IDENTIFIER

`connectionIdFor(a, b)` sorts the two UUIDs and joins them with `_`. Because  $A \rightarrow B$  and  $B \rightarrow A$  produce the same identifier, a duplicate connection cannot be created, and relationship lookup is a direct document read rather than a query.

### 9.6 `users/{uid}/notifications/{notificationId}`

Fields: `recipientId`, `actorId`, `actorName`, `actorPhotoURL`, `type` (`like` | `comment` | `connection_request` | `connection_accepted`), `targetPostId`, `read`, `createdAt`.

Modelled as a subcollection so ownership is expressed by path. Other users may create documents here — that is how notifications are delivered — but the rule requires `actorId == request.auth.uid`, preventing forged notifications.

### 9.7 Index

| Collection         | Fields                                                 | Required by                         |
|--------------------|--------------------------------------------------------|-------------------------------------|
| <code>posts</code> | <code>authorId</code> ASC, <code>createdAt</code> DESC | <code>subscribeToUserPosts()</code> |

All other queries are satisfied by Firestore's automatic single-field indexes.

## 10. Authentication design

**Method:** Firebase Authentication, email + password only. Deliberately excluded: organisational SSO, LDAP, Active Directory, SAML and third-party social providers — an isolation requirement as much as a scope decision.

| Store                              | Holds                                | Managed by      |
|------------------------------------|--------------------------------------|-----------------|
| Firebase Authentication            | UID, email, password hash            | Google          |
| Firestore <code>users/{uid}</code> | Name, headline, bio, location, photo | The application |

**The application never receives, stores or transmits a password to its own storage.** Credentials pass directly from the client SDK to Google's authentication endpoint.

### Session management

`initializeAuth()` is configured with `getReactNativePersistence(AsyncStorage)`. Firebase writes the ID token to the device's local storage and restores it at launch. `AuthContext` exposes an `initialising` flag so the UI shows a spinner rather than briefly flashing the login screen during restore.

```
onAuthStateChanged → AuthContext.user
 ↓
 RootNavigator re-renders
 ↓
signed in → main stack | signed out → AuthNavigator
```

No screen performs manual navigation after login or logout.

# 11.Security

## THE CENTRAL PRINCIPLE

Authorisation is enforced **on Google's servers** by Firebase Security Rules. They apply to every request regardless of whether it originates from the official application, a modified build, or a direct API call from a script. Client-side checks in Linkora exist only for feedback speed; they are not a security control.

### 11.1 Principles applied

| Principle               | Implementation                                                                                                                                |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Deny by default         | File terminates with <code>match /{document=**} { allow read, write: if false; }</code>                                                       |
| No test mode            | <code>allow read, write: if true</code> appears nowhere in the file                                                                           |
| Authentication required | Every rule requires <code>isSignedIn()</code>                                                                                                 |
| Ownership               | Writes compare <code>request.auth.uid</code> to the document ID or <code>authorId</code>                                                      |
| Field-level restriction | <code>diff(resource.data).affectedKeys().hasOnly([...])</code>                                                                                |
| Value-level restriction | Counter updates constrained to exactly $\pm 1$                                                                                                |
| Unbuilt features locked | <code>jobs</code> , <code>companies</code> , <code>applications</code> , <code>conversations</code> , <code>messages</code> explicitly denied |

### 11.2 Representative rules

#### PROFILE OWNERSHIP

```
allow update: if isSelf(uid)
 && onlyChanges(['displayName','headline','bio',
 'location','photoURL','updatedAt']);
```

Only the owner may write, and only presentation fields. `email` and `uid` are absent from the allow-list and therefore immutable from the client.

#### COUNTER INTEGRITY

```
allow update: if isSignedIn()
 && onlyChanges(['likeCount'])
 && (request.resource.data.likeCount == resource.data.likeCount + 1
 || request.resource.data.likeCount == resource.data.likeCount - 1);
```

Non-authors must be able to like a post, so the rule restricts *magnitude* rather than *access*.

## CONNECTION ACCEPTANCE

```
allow update: if isSignedIn()
 && request.auth.uid == resource.data.recipientId
 && resource.data.status == 'pending'
 && request.resource.data.status == 'accepted'
 && onlyChanges(['status', 'updatedAt']);
```

Four simultaneous conditions; a requester cannot accept their own request.

## NOTIFICATION AUTHENTICITY

```
allow create: if isSignedIn()
 && request.resource.data.actorId == request.auth.uid
 && request.resource.data.recipientId == uid
 && request.resource.data.read == false;
```

Cross-user writes are permitted but must be attributed truthfully.

### 11.3 Storage rules

```
match /users/{uid}/{fileName} {
 allow read: if isSignedIn();
 allow write: if isSelf(uid) && isImage() && isReasonableSize();
}
```

Every writable path is prefixed with the uploader's UID, reducing folder isolation to a single comparison. Uploads must match `image/*` and be under 5 MB.

### 11.4 Secret management

| Item                   | Handling                                                                                                                                                                |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Firebase client config | In <code>.env</code> , injected via <code>EXPO_PUBLIC_*</code> . <b>Public by design</b> — an identifier, not a credential.                                             |
| <code>.env</code>      | Git-ignored. Template <code>.env.example</code> holds placeholders only.                                                                                                |
| Service-account keys   | <b>Not present.</b> No Admin SDK. Blocked by <code>.gitignore</code> .                                                                                                  |
| Signing keystores      | Blocked by <code>.gitignore</code> ( <code>*.jks</code> , <code>*.keystore</code> , <code>*.p8</code> , <code>*.p12</code> , <code>*.key</code> , <code>*.pem</code> ). |

A mobile application cannot hold a secret — any embedded value can be extracted from the bundle. The architecture therefore assumes the client is untrusted and places all trust decisions in the security rules.

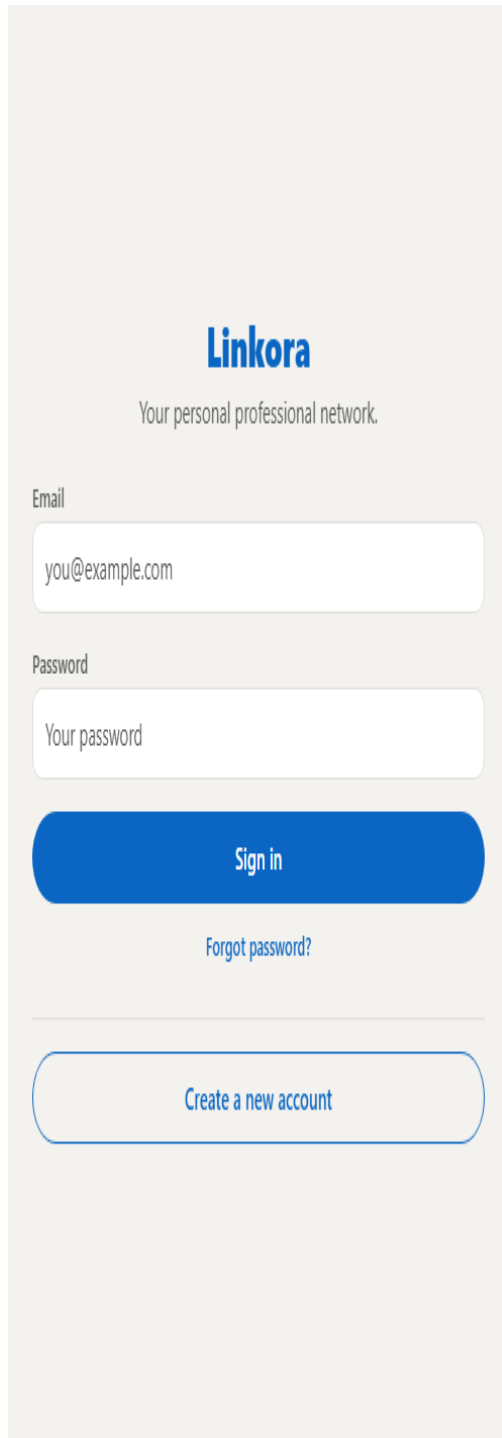
## 11.5 Known limitations

1. **Counter manipulation is limited, not prevented.** A user could repeat a permitted  $\pm 1$  write. Full prevention requires a Cloud Function (Blaze plan).
2. **All content is visible to all members.** Intended for a network application, but there is no private content tier.
3. **Registration is open.** Anyone with the application can create an account.
4. **Post deletion uses a client-side loop** and could be interrupted for a very large post.
5. **17 moderate npm advisories**, all transitive (`decode-uri-component`, `uuid`) via Expo and React Navigation. No clean remediation exists — versions are pinned upstream and `npm audit fix --force` would break SDK alignment. Neither advisory is reachable through Linkora's usage.

## 12.UI / UX — screens

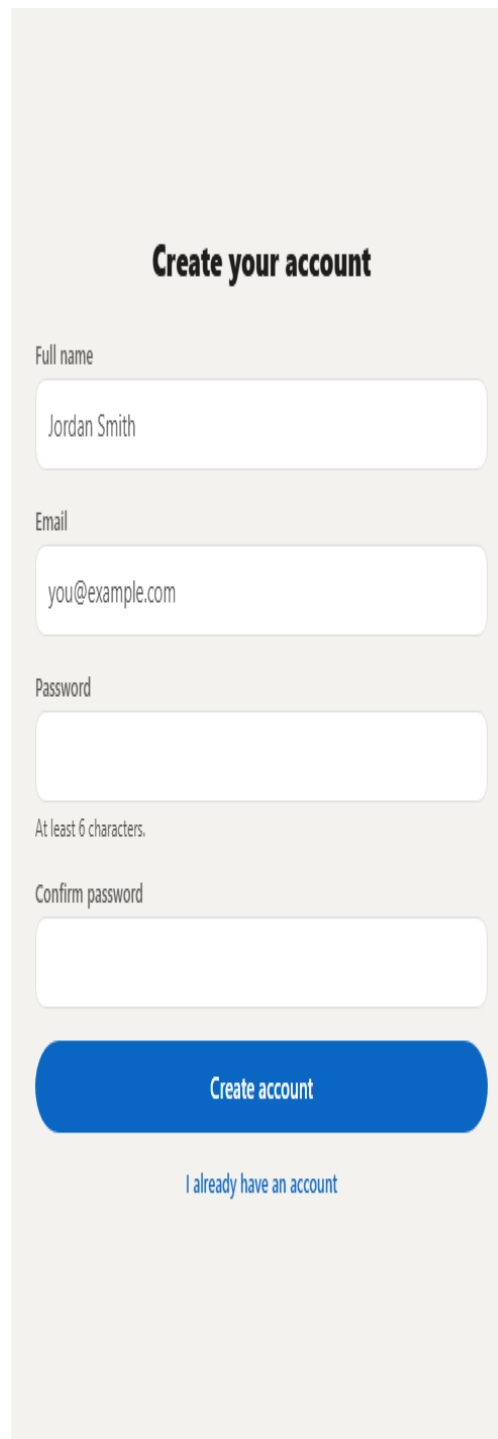
### 12.1 Verified screenshots

The two images below are **genuine screenshots of the application running**. They were captured from the web build (`npx expo export --platform web`, rendered through `react-native-web`) using a temporary throwaway Firebase configuration. The transition from Login to Register was performed by an automated click through the Chrome DevTools Protocol, and the browser console reported **zero errors or exceptions** throughout.



The login screen features the Linkora logo at the top, followed by the tagline 'Your personal professional network.' Below this is a form with an 'Email' field containing 'you@example.com' and a 'Password' field with the placeholder 'Your password'. A prominent blue 'Sign in' button is centered below the fields. A link for 'Forgot password?' is positioned directly under the sign-in button. At the bottom, a rounded rectangular button with a blue border contains the text 'Create a new account'.

Login — real render



The register screen is titled 'Create your account'. It includes a 'Full name' field with 'Jordan Smith', an 'Email' field with 'you@example.com', and a 'Password' field. Below the password field is a note 'At least 6 characters.' followed by a 'Confirm password' field. A blue 'Create account' button is centered below these fields. At the bottom, a link 'I already have an account' is displayed.

Register — real render

## WHAT THIS EVIDENCES

All of the following is platform-independent and therefore transferable to a device: successful application boot, the environment guard accepting a valid configuration, Firebase SDK initialisation, AuthContext resolving its loading state, React Navigation performing a screen transition, and correct rendering of both authentication screens.

**What it does not evidence.** The capture is the **web** render via `react-native-web`, not a physical device. Three code paths are platform-specific and were not exercised.

| Unexercised path                                     | Location                                                          | Failure symptom on a device                    |
|------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------|
| <code>getReactNativePersistence(AsyncStorage)</code> | <code>src/firebase/app.ts</code>                                  | Session does not survive an app restart        |
| <code>expo-image-picker</code> native module         | <code>CreatePostScreen</code> ,<br><code>EditProfileScreen</code> | Gallery does not open, or no permission prompt |
| <code>fetch('file:///...') → Blob</code>             | <code>src/services/storage.service.ts</code>                      | Upload fails after an image is selected        |

Real authentication and all Firestore reads and writes also remain unevidenced, as they require live credentials and published security rules. The Android bundle compiles successfully (903 modules), but compilation is not execution.

## 12.2 Interface mockups

**Statement of accuracy.** The images below are **mockups, not screenshots**. The signed-in screens require an authenticated account against live Firebase credentials, so they could not be captured. Each mockup is drawn directly from the design tokens in `src/theme/index.ts` and from each screen's `StyleSheet`, so colours, spacing, control shapes and label text correspond to the implemented code. They are presented as design references, not as evidence of a running build.

9:41



# Linkora

Your personal professional network.

EMAIL

you@example.com

PASSWORD

Your password

Sign in

[Forgot password?](#)

9:41



## Create your account

FULL NAME

Jordan Smith

EMAIL

you@example.com

PASSWORD

.....

At least 6 characters.

CONFIRM PASSWORD

.....

Create account

[I already have an account](#)

9:41



Linkora

+ Post



**Priya Sharma**

Frontend developer · React Native  
2h

Finally shipped the offline-first sync layer this week. Firestore's local cache did most of the heavy lifting. 🎉

12 likes

4 comments

♥ Liked

💬 Comment



**Arun Kumar**

Backend engineer  
5h

Notes from today's session on Firestore security rules. Deny by default, always.

🖼️ attached image

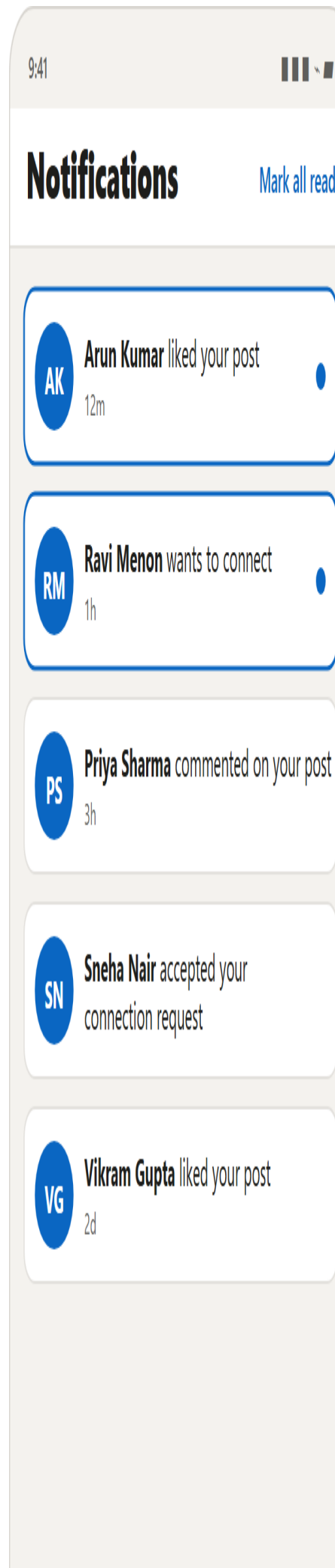
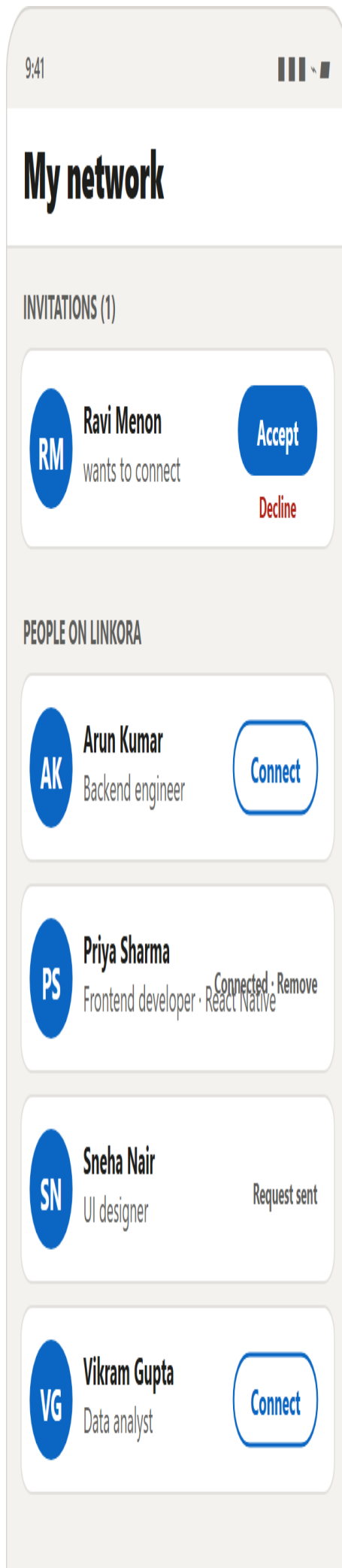
8 likes

1 comment

**Login**

**Register**

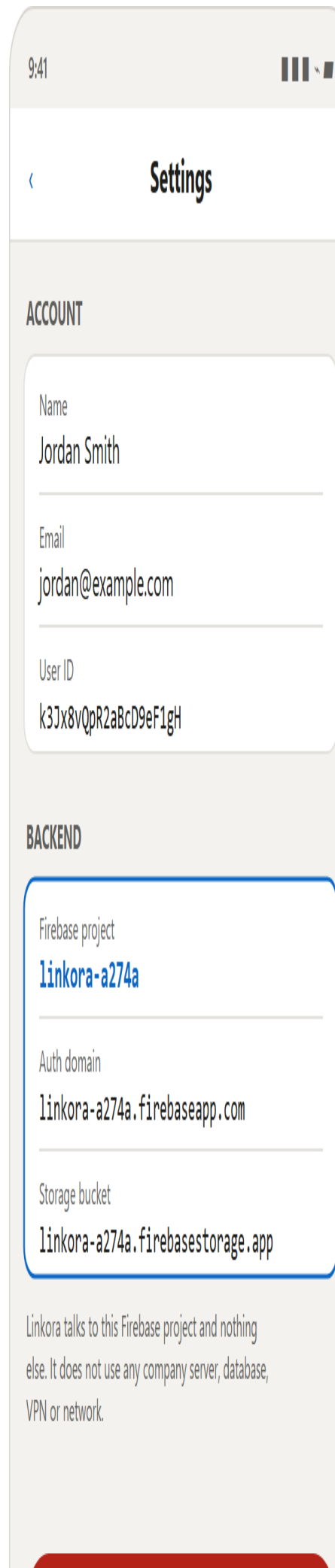
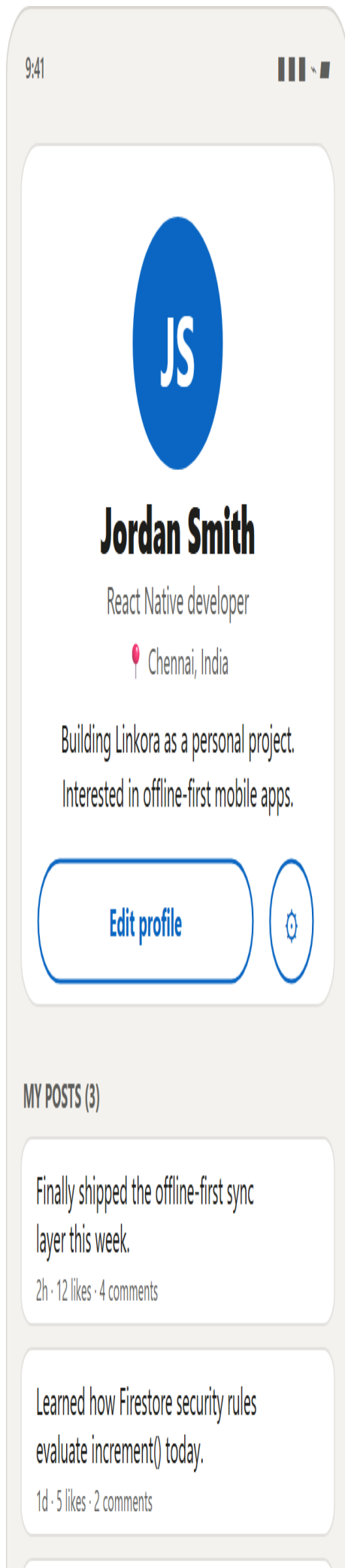
**Home feed**



**Post & comments**

**My network**

**Notifications**



**DESIGN LANGUAGE**

A single token file (`src/theme/index.ts`) defines the palette — primary `#0A66C2`, background `#F4F2EE`, surface `#FFFFFF`, border `#E3E1DD`, text `#1D1D1B` — together with a spacing scale, corner radii and font sizes. Every screen imports from it, which is why the mockups above can be generated from the source with confidence.

## 12.3 Screen inventory

**SIGNED OUT**

| Screen   | File                                 | Purpose        | Actions                                 | Data source      |
|----------|--------------------------------------|----------------|-----------------------------------------|------------------|
| Login    | <code>auth/LoginScreen.tsx</code>    | Sign in        | Sign in, reset password, go to Register | Firebase Auth    |
| Register | <code>auth/RegisterScreen.tsx</code> | Create account | Create account, return to Login         | Auth + Firestore |

**SIGNED IN — TABS**

| Screen        | File                                 | Purpose                | Actions                                               | Data source                                                                |
|---------------|--------------------------------------|------------------------|-------------------------------------------------------|----------------------------------------------------------------------------|
| Home          | <code>HomeScreen.tsx</code>          | Realtime feed          | Like, comment, open author, delete own post, new post | <code>subscribeToFeed()</code>                                             |
| Network       | <code>NetworkScreen.tsx</code>       | People and invitations | Connect, accept, decline, remove                      | <code>listOtherUsers()</code> ,<br><code>subscribeToMyConnections()</code> |
| Notifications | <code>NotificationsScreen.tsx</code> | Activity list          | Open target, mark all read                            | <code>subscribeToNotifications()</code>                                    |
| Profile       | <code>ProfileScreen.tsx</code>       | Own profile and posts  | Edit profile, settings                                | <code>useAuth()</code> ,<br><code>subscribeToUserPosts()</code>            |

**SIGNED IN — STACK**

| Screen      | File                              | Purpose | Actions                      | Data source                                                   |
|-------------|-----------------------------------|---------|------------------------------|---------------------------------------------------------------|
| Create post | <code>CreatePostScreen.tsx</code> | Compose | Attach/remove image, publish | <code>createPost()</code> ,<br><code>uploadPostImage()</code> |

|              |                                    |                          |                                 |                                                                     |
|--------------|------------------------------------|--------------------------|---------------------------------|---------------------------------------------------------------------|
| Post detail  | <code>PostDetailScreen.tsx</code>  | Post + comments          | Add comment, delete own comment | <code>getPost()</code> , <code>subscribeToComments()</code>         |
| Edit profile | <code>EditProfileScreen.tsx</code> | Edit own profile         | Change photo, save              | <code>updateUserProfile()</code> , <code>uploadAvatar()</code>      |
| User profile | <code>UserProfileScreen.tsx</code> | Another member           | Connect, accept, remove         | <code>getUserProfile()</code> , <code>subscribeToUserPosts()</code> |
| Settings     | <code>SettingsScreen.tsx</code>    | Account and backend info | Sign out                        | <code>useAuth()</code> , <code>firebaseConfig</code>                |

#### NOTABLE UX DECISION — THE SETTINGS BACKEND CARD

Settings displays the live Firebase project ID, auth domain and storage bucket. This turns the isolation guarantee into something verifiable on the device itself rather than a claim in documentation.

## 13. Firebase services used

---

Linkora consumes **no REST APIs**. All backend interaction is through the Firebase SDK.

### Authentication

| SDK function                                       | Purpose                             |
|----------------------------------------------------|-------------------------------------|
| <code>createUserWithEmailAndPassword</code>        | Registration                        |
| <code>signInWithEmailAndPassword</code>            | Login                               |
| <code>signOut</code>                               | Logout                              |
| <code>updateProfile</code>                         | Set display name on the Auth record |
| <code>sendPasswordResetEmail</code>                | Password reset                      |
| <code>onAuthStateChanged</code>                    | Session state listener              |
| <code>initializeAuth</code> / <code>getAuth</code> | Instance creation with persistence  |

### Cloud Firestore

| SDK function                               | Purpose                                                    |
|--------------------------------------------|------------------------------------------------------------|
| <code>collection</code> , <code>doc</code> | Path references                                            |
| <code>getDoc</code> , <code>getDocs</code> | One-off reads                                              |
| <code>onSnapshot</code>                    | Realtime listeners (6 sites)                               |
| <code>setDoc</code>                        | Write with a chosen ID (users, likes, connections)         |
| <code>addDoc</code>                        | Write with a generated ID (posts, comments, notifications) |
| <code>updateDoc</code>                     | Partial update                                             |
| <code>deleteDoc</code>                     | Delete                                                     |

|                                           |                                    |
|-------------------------------------------|------------------------------------|
| <code>writeBatch</code>                   | Atomic multi-write (like / unlike) |
| <code>increment</code>                    | Atomic counter                     |
| <code>serverTimestamp</code>              | Server-side time                   |
| <code>query, where, orderBy, limit</code> | Query construction                 |

## Storage

`ref · uploadBytes · getDownloadURL`

## Cloud Messaging

**NOT IMPLEMENTED** — no FCM code exists. `EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID` is present because it is part of the standard Firebase config block, not because messaging is in use.

## 14.Data flow

---

### Registration

```
RegisterScreen → register()
 → createUserWithEmailAndPassword() Firebase Auth
 → updateProfile({ displayName }) Firebase Auth
 → createUserProfile() → setDoc() Firestore users/{uid}
 → onAuthStateChanged → AuthContext → RootNavigator → MainTabs
```

### Create post

```
CreatePostScreen
 → (optional) uploadPostImage() → Storage → download URL
 → createPost() → addDoc(posts)
 → subscribeToFeed listener fires on all devices
 → HomeScreen → FlatList → PostCard
```

### Fetch posts

```
HomeScreen useEffect
 → subscribeToFeed()
 → onSnapshot(posts, orderBy createdAt desc, limit 50)
 → snapshot.docs.map(toPost) → setPosts
 → unmount: unsubscribe()
```

### Like

```
PostCard onToggleLike
 → optimistic setLikedIds()
 → likePost()
 → writeBatch: set(likes/{uid}) + update(likeCount +1) → commit()
 → createNotification('like')
 → feed listener pushes the new count
 → on failure: state rolled back, ErrorBanner shown
```

### Connection

```
NetworkScreen "Connect" → sendConnectionRequest()
 → setDoc(connections/{sortedPair}, 'pending') → notification
 → recipient's subscribeToMyConnections fires
 → "Accept" → acceptConnectionRequest() → updateDoc('accepted') → notification
```

## Image upload

```
requestMediaLibraryPermissionsAsync()
 → launchImageLibraryAsync({ quality: 0.7 })
 → file:// URI
 → fetch(uri).blob() device-local read, NOT a network request
 → uploadBytes(ref(...)) Firebase Storage
 → getDownloadURL()
 → URL saved into the Firestore document
```

## 15. Testing

**Accuracy statement.** This section reports only what was actually executed. Items marked **PENDING** have not been run.

### 15.1 Automated verification — PERFORMED

| Check                             | Command                           | Result                                           |
|-----------------------------------|-----------------------------------|--------------------------------------------------|
| TypeScript compilation            | <code>npm run typecheck</code>    | <b>Pass</b> — zero errors                        |
| Metro production bundle (Android) | <code>npm run bundle-check</code> | <b>Pass</b> — 903 modules, no resolution errors  |
| Dependency audit                  | <code>npm audit</code>            | 17 moderate, 0 high, 0 critical — all transitive |

### 15.2 Static and bundle analysis — PERFORMED

| Check                                                                                                                   | Result                                                                               |
|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Network entry points ( <code>fetch</code> , <code>axios</code> , <code>XMLHttpRequest</code> , <code>WebSocket</code> ) | <b>1 result</b> — local <code>file://</code> read in <code>storage.service.ts</code> |
| Hardcoded URLs / IPs in source                                                                                          | Only documentation links; no live endpoints                                          |
| Private IP literals in compiled bundle                                                                                  | <b>None</b>                                                                          |
| Database drivers (MongoDB, MySQL, PostgreSQL, Redis, MSSQL, Oracle)                                                     | <b>None</b>                                                                          |
| Directory / SSO clients (LDAP, Kerberos, SAML)                                                                          | <b>None</b>                                                                          |
| Telemetry SDKs (Sentry, Segment, Amplitude, Mixpanel, Bugsnag, Datadog)                                                 | <b>None</b>                                                                          |
| Embedded private keys                                                                                                   | <b>None</b>                                                                          |
| Admin SDK / service accounts                                                                                            | <b>None</b>                                                                          |
| Outbound hostnames in compiled bundle                                                                                   | Only Google Firebase endpoints                                                       |

### 15.3 Runtime boot test — PERFORMED

The application was executed as a web build with a temporary throwaway Firebase configuration and driven automatically through the Chrome DevTools Protocol.

| Check                                           | Method                                                         | Result                                          |
|-------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------|
| Application boots                               | Load exported web bundle                                       | <b>Pass</b> — renders within 3 s                |
| Environment guard accepts valid config          | <code>.env</code> with correct <code>projectId</code>          | <b>Pass</b> — no error thrown                   |
| Firebase SDK initialises                        | <code>initializeApp</code> in <code>src/firebase/app.ts</code> | <b>Pass</b> — no exception                      |
| <code>AuthContext</code> resolves loading state | <code>onAuthStateChanged</code> fires                          | <b>Pass</b> — Login replaces the splash spinner |
| Login screen renders                            | DOM text assertion                                             | <b>Pass</b> — all fields and buttons present    |
| Navigation works                                | Automated click on "Create a new account"                      | <b>Pass</b> — Register screen rendered          |
| Register screen renders                         | DOM text assertion                                             | <b>Pass</b> — all four fields present           |
| Console errors / exceptions                     | CDP <code>Runtime.exceptionThrown</code>                       | <b>Zero</b>                                     |

Additionally, `onAuthStateChanged` was probed directly in Node with a deliberately invalid API key and fired in **1 ms** with `user = null`, confirming the auth listener resolves even when the backend is unreachable.

### 15.4 Functional testing — PENDING

Testing against live data requires real Firebase credentials and completed console setup, which were not performed. **No end-to-end functional test has been executed.** The list below is the intended plan, not a record of results.

- [ ] Registration creates an Auth user and `users/{uid}`
- [ ] Login and logout
- [ ] Session persists across app restart
- [ ] Password reset email delivered
- [ ] Profile edit persists
- [ ] Avatar upload appears in Storage
- [ ] Post creation (text, and text + image)
- [ ] Feed updates in realtime on a second device
- [ ] Like increments and de-duplicates; unlike decrements

- [ ] Comment adds and increments counter
- [ ] Comment deletion by author and by post owner
- [ ] Post deletion removes subcollections
- [ ] Connection request, accept, remove
- [ ] Notifications generated for all four types; no self-notification

## 15.5 Security-rule testing — PENDING

Planned via **Firestore** → **Rules Playground**:

- [ ] Unauthenticated read of users/{uid} expect DENY
- [ ] Authenticated read of another user's profile expect ALLOW
- [ ] Write to another user's profile expect DENY
- [ ] Write email field on own profile expect DENY
- [ ] likeCount change of +1 expect ALLOW
- [ ] likeCount change of +50 expect DENY
- [ ] Create like with another user's UID as docId expect DENY
- [ ] Requester accepting own request expect DENY
- [ ] Notification with forged actorId expect DENY
- [ ] Any read of jobs / conversations expect DENY

## 15.6 Isolation testing — partially performed

| Test                                                           | Status                |
|----------------------------------------------------------------|-----------------------|
| Source and bundle scan for organisational references           | PERFORMED — CLEAN     |
| Verification that only Firebase endpoints appear in the bundle | PERFORMED — CONFIRMED |
| Run on personal Wi-Fi with organisational network unreachable  | PENDING               |
| Run with VPN disconnected                                      | PENDING               |

## 16. Error handling

### Strategy — three layers

1. **Client validation** — immediate feedback before a network call (`looksLikeEmail()`, password length, matching passwords, non-empty post).
2. **Service-layer propagation** — services do not swallow errors; they let them reach the caller.
3. **Screen-level presentation** — screens catch, translate via `describeError()`, and render `<ErrorBanner />`.

### Translation examples

| Code                                     | Message shown                                                                 |
|------------------------------------------|-------------------------------------------------------------------------------|
| <code>auth/email-already-in-use</code>   | That email address already has a Linkora account. Try signing in.             |
| <code>auth/invalid-credential</code>     | Wrong email or password.                                                      |
| <code>auth/network-request-failed</code> | Could not reach Firebase. Check that this device has internet access.         |
| <code>permission-denied</code>           | Firestore refused this operation. Your security rules do not allow it...      |
| <code>failed-precondition</code>         | Firestore needs an index for this query. Open the link in the terminal error. |
| <code>storage/unauthorized</code>        | Firebase Storage refused this upload...                                       |

Unmapped errors fall back to the raw message, then to a generic sentence — the user never sees a bare error code.

### Deliberate non-failures

| Operation                                                 | Rationale                                                                        |
|-----------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>getLikedPostIds()</code> in <code>HomeScreen</code> | A failure only means heart icons are not pre-filled. It must not break the feed. |
| <code>markNotificationRead()</code> on tap                | Navigation should proceed even if the read flag fails to write.                  |

Both are commented in the source to record the intent.

### Offline behaviour

Firestore's local cache serves recently-read data and queues writes for replay. `unavailable` and `auth/network-request-failed` have explicit user-facing messages. The application does not crash and does not discard user input.

## 17. Performance

| Technique            | Implementation                                            | Effect                                    |
|----------------------|-----------------------------------------------------------|-------------------------------------------|
| Denormalisation      | Author data copied onto each post                         | 50-post feed is 1 query, not 51           |
| Query limits         | <code>limit(50)</code> on feed, user posts, notifications | Bounded read cost and memory              |
| List virtualisation  | <code>FlatList</code> throughout                          | Only visible rows rendered                |
| Atomic increments    | <code>increment()</code> not read-modify-write            | Removes a round trip and a race           |
| Batched writes       | <code>writeBatch</code> for like + counter                | One round trip instead of two             |
| Optimistic UI        | Like state updates before the write completes             | Perceived latency $\approx 0$             |
| Deterministic IDs    | Likes by UID, connections by sorted pair                  | Eliminates pre-write queries              |
| Listener cleanup     | Every <code>useEffect</code> returns its unsubscribe      | Prevents leaks and billed reads           |
| Image compression    | <code>quality: 0.7</code> in the picker                   | Smaller uploads, faster feed              |
| Memoisation          | <code>useMemo</code> / <code>useCallback</code>           | Avoids recomputation per render           |
| Composite index      | <code>authorId + createdAt</code>                         | Server-side filtered sort                 |
| Single Firebase init | <code>getApps().length &gt; 0 ? getApp() : ...</code>     | No duplicate instances under fast refresh |

**Measured artefact.** The Android production bundle is 3.7 MB (Hermes bytecode) across 903 modules.

## 18.Security & privacy — infrastructure independence

### STATEMENT

Linkora is a personal application. It operates exclusively on the personal Firebase project `linkora-a274a` and is designed so that it **does not require or depend on organisational servers, databases, APIs, VPN, credentials or network infrastructure**.

### Enforcement in code

| Control                    | Location                                                              | Effect                                                                                              |
|----------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Single network entry point | <code>src/firebase/app.ts</code>                                      | Only file that opens a backend connection                                                           |
| No HTTP client             | Whole project                                                         | No <code>axios</code> ; exactly one <code>fetch</code> , reading a local <code>file://</code> image |
| Project allow-list         | <code>config/env.ts</code> → <code>ALLOWED_FIREBASE_PROJECT_ID</code> | Application throws and halts if <code>.env</code> names any other project                           |
| Placeholder rejection      | <code>config/env.ts</code> → <code>required()</code>                  | Refuses to start on unconfigured values                                                             |
| On-device disclosure       | <code>SettingsScreen</code>                                           | Displays live project ID, auth domain, storage bucket                                               |
| Secret exclusion           | <code>.gitignore</code>                                               | Blocks <code>.env</code> , service-account JSON, keystores, certificates                            |
| No Admin SDK               | Dependency tree                                                       | No privileged credential can exist in the client                                                    |

### External network dependencies — complete list

| Service                 | Domain                                      | Purpose                      | Ownership | Runtime? |
|-------------------------|---------------------------------------------|------------------------------|-----------|----------|
| Firebase Authentication | <code>identitytoolkit.googleapis.com</code> | Sign-in, registration, reset | Personal  | Yes      |
| Firebase token refresh  | <code>securetoken.googleapis.com</code>     | Session token renewal        | Personal  | Yes      |
| Cloud Firestore         | <code>firestore.googleapis.com</code>       | All application data         | Personal  | Yes      |

|                          |                                             |                          |          |                              |
|--------------------------|---------------------------------------------|--------------------------|----------|------------------------------|
| Firebase Storage (write) | <code>firebasestorage.googleapis.com</code> | Image upload             | Personal | Yes                          |
| Firebase Storage (read)  | <code>storage.googleapis.com</code>         | Image download           | Personal | Yes                          |
| Firebase auth domain     | <code>linkora-a274a.firebaseio.com</code>   | Auth handler             | Personal | Yes                          |
| npm registry             | <code>registry.npmjs.org</code>             | Dependency installation  | Public   | <b>No</b> — build only       |
| Expo dev server          | Local machine, port 8081                    | Code delivery to Expo Go | Local    | <b>No</b> — development only |

**Organisational infrastructure referenced: none.**

**Caveat on verification scope.** The scans above cover source code, configuration and the compiled Android bundle. They do **not** constitute a runtime network capture. A definitive runtime verification would require running the configured application behind a network monitor, which has not been performed. The static and bundle-level evidence is strong but is not a substitute for that test.

## 19. Installation guide

| Requirement                  | Necessity           | Verify with                 |
|------------------------------|---------------------|-----------------------------|
| Node.js 20+                  | Required            | <code>node --version</code> |
| npm                          | Required            | <code>npm --version</code>  |
| Expo Go on a phone           | Required            | Play Store / App Store      |
| Personal Google account      | Required            | —                           |
| Git                          | Recommended         | <code>git --version</code>  |
| Android Studio / JDK / Xcode | <b>Not required</b> | —                           |

### Steps

```
cd C:\Users\IT\Documents\Linkora
npm install
Copy-Item .env.example .env # then fill in the Firebase values
npm run typecheck
```

### Firebase console setup

1. **Project settings** → **Your apps** → register a **Web** app (the JS SDK requires a Web app even on mobile). Copy the config into `.env`.
2. **Build** → **Authentication** → **Sign-in method** → enable **Email/Password**.
3. **Build** → **Firestore Database** → create in **production mode**.
4. **Build** → **Storage** → enable in **production mode**; copy the bucket into `.env`.
5. **Firestore** → **Rules** → paste `firestore.rules`, publish.
6. **Storage** → **Rules** → paste `storage.rules`, publish.
7. Create the composite index via `firebase deploy --only firestore:indexes` or the link in the runtime error.

`google-services.json` and `GoogleService-Info.plist` are **not required**. The JS SDK uses the Web app configuration on all platforms.

## 20. Running the application

| Command                              | Purpose                                                              |
|--------------------------------------|----------------------------------------------------------------------|
| <code>npm start</code>               | Start the Expo development server                                    |
| <code>npm run android</code>         | Open on a connected Android device or emulator                       |
| <code>npm run ios</code>             | Open in the iOS simulator (macOS only)                               |
| <code>npm run web</code>             | Open in a browser                                                    |
| <code>npm run typecheck</code>       | TypeScript check                                                     |
| <code>npm run bundle-check</code>    | Produce a production bundle                                          |
| <code>npx expo start --clear</code>  | Start with cleared cache (required after editing <code>.env</code> ) |
| <code>npx expo start --tunnel</code> | Tunnel mode when the local network blocks device-to-device traffic   |

**On a physical device:** run `npm start`, connect the phone to the same Wi-Fi, and scan the QR code with Expo Go (Android) or the Camera app (iOS).

**Version control note.** This project is **not currently a Git repository** — Git is not installed on the development machine. When initialising it, use a personal Git identity and a private personal remote, and confirm `.env` is absent from `git status --porcelain` before the first commit.

## 21. Deployment

Linkora has no server component. Deployment consists of publishing security rules and distributing the application binary.

### Rules deployment

```
npm install -g firebase-tools
firebase login
firebase use linkora-a274a
firebase deploy --only firestore:rules,firestore:indexes,storage
```

Rules must be deployed **before** distributing an application version that depends on them.

### Android — cloud build

```
npm install -g eas-cli
eas login
eas build:configure
eas build --platform android --profile preview # APK
eas build --platform android # AAB for Play Store
```

EAS Build uploads source to Expo's servers — the only third-party service in this project's lifecycle. For a personal project with no embedded secrets the exposure is low, but it should be a conscious choice. `EXPO_PUBLIC_*` variables must be configured in `eas.json` or the EAS dashboard, as the build server has no access to the local `.env`.

### Android — local build

Requires JDK 17 and Android Studio, neither installed on the development machine:

```
npx expo prebuild --platform android
cd android
./gradlew assembleRelease # APK
./gradlew bundleRelease # AAB
```

A release keystore is required and must be backed up outside version control — losing it prevents publishing further updates to the same application identity.

### iOS

`eas build --platform ios` requires a paid Apple Developer account (USD 99/year). For personal use, Expo Go on a physical iPhone is sufficient.

## 22.Future enhancements

These are **proposals**, not implemented functionality.

### Completing the designed model

| Enhancement                     | Notes                                                                                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Direct messaging                | <code>conversations/{id}/messages/{id}</code> with a <code>members</code> array, mirroring the connections pattern. Rule placeholders already exist. |
| Jobs and applications           | <code>jobs</code> , <code>companies</code> , <code>applications</code> collections. Rule placeholders already exist.                                 |
| Push notifications              | Requires a development build plus <code>expo-notifications</code> and a Cloud Function to dispatch messages.                                         |
| Search                          | <code>findUserByEmail()</code> already exists and needs only a UI. Substring search would require an external index.                                 |
| Post editing / account deletion | Both already permitted by the rules; only UI is missing.                                                                                             |

### Engineering improvements

| Enhancement       | Rationale                                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------|
| Automated tests   | <code>jest-expo</code> + <code>@testing-library/react-native</code> ; rules tests via the Firebase emulator |
| Cloud Functions   | Server-authoritative counters, cascading deletes, notification fan-out                                      |
| Pagination        | Cursor-based infinite scroll to replace the fixed <code>limit(50)</code>                                    |
| Image caching     | <code>expo-image</code> for disk caching and progressive loading                                            |
| Offline indicator | Explicit UI state for queued writes                                                                         |
| CI pipeline       | Automated <code>typecheck</code> and <code>bundle-check</code> on every commit                              |

### Product ideas

Advanced job recommendations · improved search ranking · profile verification · AI-assisted career guidance · richer notification preferences · usage analytics on personally-owned infrastructure.

## 23. Conclusion

---

Linkora demonstrates that a functional, secure, cross-platform social application can be built without operating any server infrastructure. By using Firebase as a serverless backend and enforcing authorisation through server-side security rules, the project achieves a security posture that does not depend on the client being trustworthy — a property a naive client-validated design cannot claim.

The implemented feature set — authentication, profiles, a realtime feed, posts with images, likes, comments, connections and notifications — covers the core interaction loop of a professional network. Features that were designed but not built are documented as such throughout, and their Firestore collections are explicitly denied in the security rules so that incomplete work cannot become an unsecured surface.

The architectural constraint that shaped this project most is infrastructure independence. Confining all network access to a single module, hardcoding a project allow-list that halts the application on mismatch, and surfacing the live backend identity on the Settings screen together turn a policy statement into a verifiable property. Static analysis of both source and the compiled bundle confirms that the application's only network destinations are the Firebase endpoints of one personally-owned project.

The principal limitations are honest ones: no automated test suite, counter integrity that is bounded rather than guaranteed without Cloud Functions, and several designed features still unimplemented. Each is documented with the specific work required to address it.

## 24.Viva / presentation questions

---

### Q1.What is React Native?

A framework for building mobile applications in JavaScript or TypeScript that render as genuine native platform widgets, rather than as a web page inside a wrapper. One codebase targets both Android and iOS.

### Q2.Why did you choose React Native?

A single codebase for both platforms, a large ecosystem, and — combined with Expo — the ability to develop and run on a physical device without Android Studio, Xcode or a JDK. The development machine used for this project has none of those installed.

### Q3.What is Expo and why use it?

A toolchain and runtime around React Native. Expo Go loads the application over Wi-Fi from the development server, removing the entire native build setup. The trade-off is that native-only modules — including Firebase Cloud Messaging — are unavailable, which is directly why push notifications are not implemented.

### Q4.Why Firebase?

It provides authentication, a realtime database and file storage as managed services, with authorisation enforced server-side by security rules. This removes the need to write, host and secure a backend, while still keeping trust decisions off the client.

### Q5.What is Firestore?

A NoSQL document database. Data is stored as documents containing fields, grouped into collections, which may in turn contain subcollections. It supports realtime listeners that push changes to connected clients.

### Q6.How does authentication work in your app?

Firebase Authentication with email and password — `createUserWithEmailAndPassword` and `signInWithEmailAndPassword` in `src/services/auth.service.ts`. Google stores the password hash; the application never sees or stores a password. The returned ID token is persisted to `AsyncStorage` and attached to every subsequent request, which is what populates `request.auth` in the rules.

### Q7.How does your app store data?

In Cloud Firestore: `users`, `posts` (with `likes` and `comments` subcollections), `connections`, and `notifications` as a subcollection of `users`. Images live in Firebase Storage and are referenced from Firestore by download URL.

### Q8.How do users connect?

A document at `connections/{id}` where the ID is the two user IDs sorted and joined with an underscore. The requester creates it with status `pending`; only the recipient may change it to `accepted`. The deterministic ID makes duplicate connections structurally impossible.

### Q9.How does messaging work?

**It is not implemented.** There is no messaging screen, service or route. The `conversations` and `messages` collections exist in the planned data model and are explicitly denied in the security rules so the unbuilt feature cannot become an unsecured collection.

### Q10.How do Security Rules work?

They are a declarative policy evaluated on Google's servers for every request. Because they are not part of the application, they apply even to a modified client or a direct API call. Linkora denies everything by default, requires authentication throughout, compares `request.auth.uid` against the target document, and restricts which fields — and for counters, which values — a write may produce.

### Q11.How is the application secured?

Four layers. Authentication is required for every operation. Ownership is enforced by comparing the caller's UID to the document. Field-level rules limit even permitted writes. And the signed-out UI never mounts the signed-in screens. The first three are server-enforced and cannot be bypassed by editing the app.

### Q12.Is your Firebase API key exposed? Is that a problem?

It is embedded in the bundle, and that is expected. Firebase client configuration is a public project identifier, comparable to a postal address — it grants no access by itself. Access is decided entirely by the security rules. The `EXPO_PUBLIC_` prefix exists precisely to signal that the value is visible. Genuine secrets, such as service-account keys, are never placed in a mobile application because any embedded value can be extracted.

### Q13.How does your app handle errors?

`src/utls/errors.ts` translates Firebase error codes into actionable sentences; screens catch errors and render a consistent `ErrorBanner`. Listeners receive an `onError` callback. Two operations deliberately swallow errors — pre-filling like icons and marking a notification read on tap — because failing loudly there would degrade the experience more than failing silently.

### Q14.How does your application communicate with Firebase?

Through the Firebase JS SDK only. There is no REST API and no custom endpoint. Screens call service functions, services use the SDK, and the SDK issues authenticated HTTPS requests — all flowing through one initialisation module, `src/firebase/app.ts`.

### Q15.Why didn't you use a traditional backend?

A conventional API tier exists mainly to hold credentials and enforce authorisation away from the client. Firebase Security Rules perform that authorisation on Google's servers. A middle tier would add hosting cost, deployment work and another attack surface without adding a guarantee the rules do not already provide. The genuine limitation is that server-authoritative logic — such as tamper-proof counters — would require Cloud Functions.

### Q16.What are the limitations of your project?

No messaging, jobs or push notifications. No automated tests. Like and comment counters are bounded to  $\pm 1$  per write but not fully tamper-proof without Cloud Functions. All content is visible to all signed-in members. Registration is open. Post deletion iterates subcollections from the client and could be interrupted for a very large post.

### Q17.What would you improve in future?

Implement messaging and jobs using the collection placeholders already reserved in the rules; add Cloud Functions for authoritative counters and cascading deletes; add a test suite with the Firebase emulator; replace the fixed feed limit with cursor pagination; and migrate to a development build to enable push notifications.

### Q18.What is the difference between `addDoc` and `setDoc`?

`addDoc` lets Firestore generate the document ID, used for posts, comments and notifications where the ID carries no meaning. `setDoc` is used where the ID is meaningful and enforces uniqueness — likes keyed by the liker's UID, and connections keyed by the sorted user pair.

#### Q19. Why `serverTimestamp()` rather than `new Date()`?

So ordering uses Google's clock rather than the device's. A phone with an incorrect date could otherwise pin its post permanently to the top of the feed. A consequence is that `createdAt` is briefly `null` in the local snapshot before the server value arrives, which is why `timeAgo()` returns "just now" for a null value.

#### Q20. What is a `writeBatch` and why is it necessary?

It groups writes so they succeed or fail together. Liking a post writes two documents — the like and the incremented counter. Without a batch, one could succeed while the other failed, leaving `likeCount` permanently wrong.

#### Q21. How do you prevent a user liking a post twice?

The like document's ID is the liker's UID: `posts/{postId}/likes/{uid}`. A second like overwrites the same document, so duplication is structurally impossible. It also reduces the rule to a single identity comparison and removes the need for a check query before writing.

#### Q22. How do you know the app only talks to your own Firebase project?

Four ways. All network access is confined to `src/firebase/app.ts`. `assertPersonalFirebaseProject()` halts the application if `.env` names any project other than `linkora-a274a`. The Settings screen displays the live project ID on the device. And extracting hostnames from the compiled Android bundle returns only Google's Firebase endpoints.

#### Q23. What did you actually test, and what did you not?

**Verified by execution:** TypeScript compiles with zero errors; the Android bundle builds successfully (903 modules); the dependency audit shows 17 moderate transitive advisories and no high or critical ones; and static plus bundle analysis found no organisational infrastructure, no database drivers, no telemetry SDKs and no embedded keys. **Not yet performed:** functional testing on a device and security-rule simulation, both of which require a configured `.env` and completed Firebase console setup.